EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

After a valid move, the program should check HasWon() before BoardFull() .

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

DO NOW — RECALL

Noughts & Crosses · Part 4 — Complete and test

Name: _____ Class: _____ Date: _____

1. What does MakeMove() return after a successful move?

2. For a 3 × 3 C# array, what does board.GetLength(0) return?

3. In Triple(7) , identify the argument. In Triple(int number) , identify the parameter.

4. A 3 × 3 game board is displayed using repeated row-printing statements. Explain how nested loops improve the code and what each loop controls.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

true.

2. Model answer

3 — the number of rows.

3. Model answer

Argument: 7. Parameter: number.

4. Model answer

An outer loop visits rows 0–2; an inner loop visits columns 0–2. The same operation is reused instead of duplicated, making the code easier to maintain.

By the end, you can…

• complete HasWon() for rows, columns and diagonals;

• detect a draw using BoardFull() ;

• finish, test and debug a complete playable game.

Vocabulary — know these terms

Recall the term — Boolean

Recall the term — Condition

Recall the term — State

Recall the term — Game loop

Recall the term — Test case

Recall the term — Trace

Recall the term — Debug

Recall the term — Draw

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Complete the game loop
Draw the board, ask the current player for a move, call MakeMove() , then check for a win or draw.
Switch player only after a valid move that did not end the game.

Question 4

Partner challenge: play and test
Pair up: one pupil plays X and the other O. Run your own C# program, make moves 1–9 and swap who goes first for a second complete game.
Evidence: for each targeted test, record the input or board state, expected result, actual result and pass/fail. Fix and retest any failure. Test all eight winning lines using prepared moves or board states; ordinary matches alone may not cover them.
If your online C# runner does not support repeated keyboard input, run the program in a local C# console or use planned input sequences for testing.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Finish all 8 winning lines
Continue your Lesson 015 program.
Complete HasWon(string player) so it detects all 3 rows, all 3 columns and both diagonals.

Question 2

Detect a draw
Create BoardFull() .
Return false as soon as a square contains neither X nor O. Return true only if every square is occupied.

NEW LEARNING

Read, annotate and explain

Eight ways to win

3 rows + 3 columns + 2 diagonals = 8 win lines. To finish the game, add HasWon(player) , BoardFull() and the game loop. Test every line, not just the top row.

HasWon() checks 8 lines

Each winning line needs three comparisons joined with && . The possible lines are joined with || .

BoardFull() looks for any free square

If even one square is free, the board is not full.

Finish Main()

Part 3 already handled valid moves and switching. Today we add the stopping conditions.

Can two pupils actually play it?

After completing the code, play against a partner and compare expected with actual results. Use planned board positions to test all eight winning lines.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Build BoardFull()

Use nested loops

```
for (int row = 0; row < board.GetLength(0); row++)
{
    for (int column = 0; column < board.GetLength(1); column++)
    {
    }
}
```

One free square means not full

```
if (board[row, column] != "X" &&
    board[row, column] != "O")
{
    return false;
}
```

The function can stop immediately when it finds a free square.

No free square was found

```
return true;
```

This line is reached only after every position has been checked.

Check draw after checking win

```
if (HasWon(currentPlayer))
{
    // win
}
else if (BoardFull())
{
    // draw
}
```


WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Complete one diagonal

Player X wins on the main diagonal when all three diagonal positions contain X.

```
if (board[0,0] == ? &&
    board[1,1] == ? &&
    board[2,2] == ?)
{
    return true;
}
```

Method / working:

2. Spot a free square

A square is free when it contains neither X nor O.

Method / working:

3. Put the actions in order

A valid move has just been made. What should happen next?

Method / working:

4. Choose the test

No winning line exists and every square is occupied.

Method / working: